EEE96005. ARTIFICIAL INTELLIGENCE

1. This question is about declarative programming and Prolog.

   Important: any answer to any part that doesn't use comments to illustrate how the predicate is queried, and to explain adequately what each line of code does, will get zero marks.

   a) Given two lists of unordered elements, of unknown and possibly different lengths, write a Prolog predicate that performs a pairwise comparison of the elements of the list, and counts the number of pairwise matches.

   For example the pairwise comparison of `[1,2,3,4]` with `[1,2,3,5,7,9]` returns 3, as does the pairwise comparison of `[1,2,3,5,7,9]` with `[1,2,3,4]`.

   [6]

   b) Given two lists of different lengths, one known to be shorter than the other, write a Prolog predicate that computes the largest pairwise comparison of the shorter list with a sub-list of the longer list.

   For example, the best pairwise comparison of `[1,2,3,4]` with `[7,1,2,3,7]` is 3.

   [4]

   c) Given a list of ordered elements, write a Prolog predicate that computes the set of pairwise preferences over the list.

   For example, for the list `[x,y,z]`, the set of pairwise preferences is: `{(x,y),(x,z),(y,z)}`.

   [4]

   d) A matrix in Prolog can be represented using a list of lists; for example the 2x2 identity matrix can be represented as `[ [1, 0], [0, 1] ]`.

   Write a Prolog predicate that generates the identity matrix for a matrix of size *N*.

   [6].

```
/*
** (a)
** ?- pw( +L1, +L2, 0, -Result ).
** L1 and L2 are the lists to be compared
** 0 is the accumulator
** Result is nominative determinism
*/
pw( [], _, Result, Result ) :-                  %base case, one list finished
        !.                                      %don't try others
pw( _, [], Result, Result ) :-                  %base case, other list finished
        !.                                      %don't try others
pw( [H|T1], [H|T2], SoFar, Result ) :-          %heads match
        !,                                      %don't try last case
        Further is SoFar + 1,                   %increment match count
        pw( T1, T2, Further, Result ).          %recurse on tail
pw( [_|T1], [_|T2], SoFar, Result ) :-          %heads don't mstch
        pw( T1, T2, SoFar, Result ).            %recurse on tail

/*
** (b)
** ?- bestpw( +LongerList, +ShorterList, 0, -Result ).
** Third argument is the accumulator
*/
bestpw( Long, Short, Result, Result ) :-        %base case
        length( Long, Len1 ),                   %length of longer lidt
        length( Short, Len2 ),                  %length of shorter list
        Len2 > Len1,                            %more in shorter than longer
        !.                                      %don't try next

bestpw( [H|T1], Short, SoFar, Result ) :-       %process case
        pw( Short, [H|T1], 0, Temp ),           %pairwise comparisomn on sublist
        max( SoFar, Temp, Further ),            %see is greater than already
        bestpw( T1, Short, Further, Result ).   %recurse on tail

max( X, Y, X ) :- X > Y, !.                     %max of two numbers
max( _, Y, Y ).                                 %if it isn't X, it is Y

/*
** (c)
** ?- pwpref( +OrderedList, [], -Result ).
** Second argument is the accumulator
*/
pwpref( [_], Result, Result ) :-                %stop when only one element in list
        !.                                      %don't try next
pwpref( [H|T], SoFar, Result ) :-               %process head
        setof( (H,X), member(X,T), Prefs ),     %generate preference over each member of tail
        append( SoFar, Prefs, Further ),        %append to what we already have
        pwpref( T, Further, Result ).           %recursive call

/*
** (d)
** ?- id_matrix( +N, -IdentityMatrix ).
*/
id_matrix( N, IDM ) :-                          %basic call
        for_each_row( 1, N, IDM ).              %for each row

for_each_row( M, N, [] ) :-                     %base case
        M > N, !.                               %rows more than size, stop, don't backtrack
for_each_row( M, N, [Row|L] ) :-
        Pre1 is M-1,                            %how many leading zeros
        Post1 is N-M,                           %how many trailing zeros
        gen0s( Pre1, Pre1s ),                   %generate front 0s
```

```
        gen0s( Post1, Post1s ),                 %generate back 0s
        append( Pre1s, [1|Post1s], Row ),       %append front+[1]+back
        M1 is M + 1,                            %increment for next row
        for_each_row( M1, N, L ).               %recurse

gen0s( 0, [] ) :-                               %finished
        !.                                      %no backtracking
gen0s( N1, [0|L] ) :-                           %add one zero
        N is N1 - 1,                            %decrement
        gen0s( N, L ).                          %recurse
```

2. This question is about search algorithms and the General Graph Search Engine (GGSE).

a) Explain, with the help of an example, how a state space can be represented in Prolog, so that it can be used for planning and problem-solving with the GGSE.

[4]

b) There are two professors, three students, and a boat on one bank of a river. The boat can hold at most one professor or (exclusively) two students, and at least one person must be in the boat to make it move. They all want to get to the opposite bank.

Formulate this river-crossing problem as a state space, so that it could be solved with the GGSE.

[6]

c) Compare and contrast breadth first, depth first, uniform cost, Beam, best first and A* search algorithms according to four criteria for evaluating different search algorithms.

[3]

d) Give, in Prolog, an extension of the GGSE for iterative-widening, iterative-deepening Beam search. Assume, on each iteration, that both the beam width and the depth limit are incremented.

Briefly explain how a path is selected for expansion (`choose/3`) and how a 'larger' graph is constructed (`add_to_paths/3`) for recursive search.

[5]

e) State, with justification, for what kind of expected search space this algorithm might be suitable.

[2]

```
/*
** (a)
*/
state -- compound term with functor, or tuple
node -- compound term with functor, or tuple, including state and possibly more information,
for example, f-, g- and h-cost, rule used to reach state, etc
path -- list of nodes
graph -- list of paths

Example: two water jugs problem

state -- state(Integer1,Integer2)
        where Integer 1 represents water in 5 pint jug, Integer2 represents water in 7 point jug
e.g. initial state is state(0,0)

node -- node( state(0,0), initial )

path -- [ node(state(5,0), fill5), node(state(0,0), initial) ]

graph -- [
          node(state(5,0), fill5), node(state(0,0), initial),
          node(state(0,7), fill7), node(state(0,0), initial)
         ]

The we want to define the goal and the state-change rules

/*
** (b)
*/
State   (
        Prof on Left Bank : [0..2],
        Stud on Left Bank : [0..3],
        Prof on Right Bank : [0..2],
        Stud on Right Bank : [0..3],
        Bank of Boat : [l,r]
        )

Initial State: state( 2, 3, 0, 0, l )

Goal state: state( 0, 0, 2, 3, r )

State change rules:

state_change( move1prof, state(PL, SL, PR, SR, B), state(PLnew, SL, PRnew, SR, OB) ) :-
        move1prof( PL, PR, PLnew, PRnew, B, OB ).

state_change( move1stud, state(PL, SL, PR, SR, B), state(PL, SLnew, PR, SRnew, OB) ) :-
        move1stud( SL, SR, SLnew, SRnew, B, OB ).

state_change( move2stud, state(PL, SL, PR, SR, B), state(PL, SLnew, PR, SRnew, OB) ) :-
        move1stud( SL, SR, SLnew, SRnew, B, OB ).


move1prof( PL, PR, PLnew, PRnew, l, r ) :-
        PL > 0,
        PLnew is PL - 1,
        PRnew is PR + 1.
move1prof( PL, PR, PLnew, PRnew, r, l ) :-
        PR > 0,
        PLnew is PL + 1,
        PRnew is PR - 1.
```

```
move1stud is like move1prof

move2stud( SL, SR, SLnew, SRnew, l, r ) :-
        SL > 1,
        SLnew is SL - 2,
        SRnew is SR + 2.
move2stud( SL, SR, SLnew, SRnew, r, l ) :-
        SL > 1,
        SLnew is SL + 2,
        SRnew is SR - 2.

/*
** (c)
*/
Given:
        b = branching factor
        d = depth of solution
        m = maximum depth of tree
        w = width of the beam

                optimality      completeness    space-complex   time-complex
breadth         yes1            yes2            O(b^d)          O(b^d)
depth           no3             no              O(b*m)          O(b^m)
uniform cost    yes             yes4            O(b^d)          O(b^d)
Beam            no              no              O(w*m)          O(w*d)
best first      no              no              O(b^m)          O(b^m)
A*              yes             yes             exponential in the length of the path

/*
** (d)
*/

/*
** wrapper to increment beam-width and depth-limit
*/
iw_id_search( Paths, SolnPath, Width, Depth ) :-
        search( Paths, SolnPath, Width, Depth ).
iw_id search( Paths, SolnPath, Depth ) :-
        Depth1 is Depth + 1,
        Width1 is Width + 1,
        iw_id_search( Paths, SolnPath, Width1, Depth1 ).

/*
** Extension to search/3, second clause
*/
search( Graph, SolnPath, Width, Depth ) :-
       choose( Path, Graph, GraphMinus ),
       pathdepth( Path, PathDepth ),                    %depth is length-1
       PathDepth = Depth,
        !,                                              %Note the 'cut' here
       search( GraphMinus, SolnPath, Width, Depth ).

/*
** Extension to add_to_paths for beam search
*/
choose: select head
add to paths: insert in order, and then prune.

/*
** (e)
*/
Search spaces with a large branching factor and many candidate solutions at low depths
```

3. This question is about graph representation, the A* algorithm and heuristics.

a) Given a graph explicitly defined as $G_S = \langle N, E, R \rangle$, give an inductive definition of the paths in the graph rooted on a node $S \in N$.

[2]

b) Given a graph explicitly defined as $G' = \langle S, op \rangle$, give an inductive definition of the paths in the graph.

[2]

c) State what condition needs to hold for the paths produced by a graph G_S to be the same as the paths produced by G'.

[2]

d) Explain how the A* search algorithm expands the paths in a graph defined as $G' = \langle S, op \rangle$.

Prove that the solution path returned by the A* search algorithm is optimal.

[4]

e) In the context of A* search, explain the conditions for a heuristic to be usable and admissible.

Explain why these conditions might make one heuristic perform "better" than another.

Comment on this explanation with respect to optimality and complexity.

[6]

f) Explain how heuristics can be used in search algorithms for 2-player games.

[4]

/* ** (a) */

$$P_G = \bigcup_{i=0}^{\infty} P_i$$
$$P_0 = \{[S]\}$$
$$P_{i+1} = \{[n \mid p] \mid \exists p \in P_i.(frontier(p), e, n) \in R\}$$

/* ** (b) */

$$P_{G'} = \bigcup_{i=0}^{\infty} P'_i$$
$$P'_0 = \{[S]\}$$
$$P'_{i+1} = \{[n \mid p] \mid \exists p \in P'_i.\exists op \in Op.op(frontier(p)) = (n, e)\}$$

/* ** (c) */

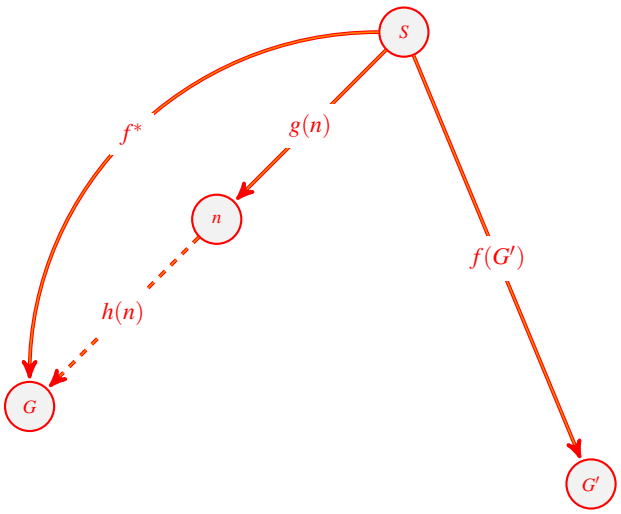
Expands that path in whichever P_i has been partially or completely filled in, whose frontier node's h-cost is least

/* ** (d) */

- Let G be an optimal goal state, with path cost f^*
- Let G" be a sub-optimal goal state with path cost $f(G')$. Then

$$f(G') = g(g') + h(G') = g(G') + 0 = g(G')$$

- Suppose A* returns G' as its (sub-optimal) solution, so $f(G') > f^*$, so $g(G') > f^*$.
- Now consider a node n on the frontier node on a path leading to G



$f^* \geq f(n)$	h is admissible and the path cost is monotonically increasing
$f(n) \geq f(G')$	otherwise n would have been selected for expansion before G'
$f^* \geq f(G')$	transitivity
$f^* \geq g(G')$	

- This is a contradiction

- So either G' was optimal, or A* never expands a sub-optimal goal

/* ** (e) */

If G is a goal state, if and only h(G) = 0

monotonic, or made monotonic by pathmax function

never overestimates actual cost of getting to goal

- Expanding the node with the lowest f-cost
  - If we estimate $h_o > h_{actual}$, so $g(n) + h_o > g(n') + h(n')$, then we might find a path from S to a sub-optimal goal G' through n'
  - If we estimate $h_u < h_{actual}$, at some point $g(n) + h_u < g(n') + h(n')$, so n will be expanded
  - If we always under-estimate, at some point we will expand the solution path ending in G before any solution path ending in G'

- Imagine drawing a histogram with increasing f-cost along the x-axis
  - Then map all the nodes onto the histogram
  - A* will expand all those nodes to the left of the f^* bar
  - Therefore the 'trick' is to get as many nodes to the right of the f^* bar ...
  - ... without ever over-estimating the h-cost

- Make $h(n)$ as large as possible (without over-estimating)
  - A* expands all nodes for which $f(n) < f^*$
  - So A* expands all nodes for which $h(n) < f^* - g(n)$
  - Therefore if $h_2(n) \geq h_1(n)$, for all n, h_2 may expand fewer nodes

Doesn't matter which heuristic, A* is still optimal

Big-O does worst case complexity, actual complexity significantly reduced by concise by accurate knowledge representation and effective heuristic computation (as close as possible to actual cost without ever over-estimating)

/* ** (f) */

evaluate leaf nodes in minimax to fixed ply

help prune the search space in alphabeta search

4. This question is about knowledge representation and reasoning with **KE**. Proofs in **KE** should be properly annotated to show the inference steps, i.e. showing the inference rule used and line number(s) of the premise(s),

a) Consider the following statements:

1. *Every maximal being is more-provable than every impossible world.*
2. *There is an inter-dimensional travel method that is more-provable than every mathematician.*
3. *Every inter-dimensional travel method is an impossible world.*
4. *More-provable is transitive.*
5. *Thör is a maximal being.*
6. *Gödel is a mathematician.*

i) Express these statements as formulas of Predicate Logic.

ii) Convert these formulas into Horn Clauses.

iii) Using resolution and showing all unifiers, prove by negation that:

Thör is more-provable than Gödel

iv) Convert the Horn Clauses into a Prolog program.

[8]

b) Consider the following argument:

If spring is sprung and the grass is green, then either I wonder where the birds are or I see that the birds are on the wing.
It is not the case that if the grass is green, then I wonder where the birds are.
Therefore, if I do not see that the birds are on the wing, then spring is not sprung.

Rewrite this argument in propositional logic, then use the calculus **KE** to prove that the argument is valid.

[6]

c) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three ($n = 3$) assertions:

1. $p \rightarrow (q \vee r)$
2. $p \rightarrow \neg s$
3. $s \leftrightarrow q$

Suppose the candidate channels her inner Nike and accepts every premise: should she accept or reject the assertion $p \rightarrow r$, or doesn't it matter?

(Hint: use **KE** for model building. If the proof has at least one open branch, then this implies a consistent sequence of answers.)

[6]

/* ** (a) */

(i)

- $\forall x.maxbeing(x) \rightarrow \forall y.impossworld(y) \rightarrow moreprov(x,y)$
- $\exists x.interdim(x) \wedge \forall y.maff(y) \rightarrow moreprov(x,y)$
- $\forall x.interdim(x) \rightarrow impossworld(x)$
- $\forall x.\forall y.\forall z.(moreprov(x,y) \wedge moreprov(y,z)) \rightarrow moreprov(x,z)$
- $maxbeing(thor)$
- $maff(godel)$

(ii)

$\neg maxbeing(x1) \vee \neg impossworld(y1) \vee moreprov(x1,y1)$	p1
$interdim(c)$	p2
$\neg maff(y2) \vee moreprov(c,y2)$	p3
$\neg interdim(x3) \vee impossworld(x3)$	p4
$\neg moreprov(x4,y4) \vee \neg moreprov(y4,z4) \vee moreprov(x4,z4)$	p5
$maxbeing(thor)$	p6
$maff(godel)$	p7

(iii)

	$\neg moreprov(thor,godel)$		$\neg$conc
1	$\neg moreprov(thor,y4) \vee \neg moreprov(y4,godel)$	$\{x4 \mapsto thor\, z4 \mapsto godel\}$	($\neg$conc, p5)
2	$\neg maxbeing(thor) \vee \neg impossworld(y1) \vee \neg moreprov(y1,godel)$	$\{x1 \mapsto thor, y1 \mapsto y4\}$	(1, p1)
3	$\neg impossworld(y1) \vee \neg moreprov(y1,godel)$		(2, p6)
4	$\neg interdim(x3) \vee \neg moreprov(x3,godel)$	$\{x3 \mapsto y1\}$	(3, p4)
5	$\neg moreprov(sk,godel)$	$\{x3 \mapsto c\}$	(4, p2)
6	$\neg maff(ralph)$	$\{y2 \mapsto godel\}$	(5, p3)
7	$\perp$		(6, p7)

/*

(iv)

```
moreprov( X, Y ) :-
        maxbeing( X ),
        impossworld( Y ).

moreprov( sk, Y ) :-
        maff( Y ).

moreprov( X, Z ) :-
        moreprov( X, Y ),
        moreprov( Y, Z ).

impossworld( X ) :-
        interdim( X ).

interdim( sk ).
maxbeing( thor ).
maff( godel ).

/*
** (b)
*/

Let
s = spring is sprung
g = the grass is green
w = I wonder where the birds are
```

```
b = I see the birds are on the wing

| ?- ke( [(s&g)->(w+b), -(g->w), -(-b -> -s)] ).
   1     s & g -> w + b
   2     - (g -> w)
   3     - (- b -> - s)
   4     alpha 2         g
   5     alpha 2         - w
   6     alpha 3         - b
   7     alpha 3         - (- s)
   8     dne   7         s
   9     br-1  1              - (s & g)
  10          beta  9,8       - g
              close 10,4
  11     br-2  1              s & g
  12          beta  1,11      w + b
  13          beta  12,5      b
              close 13,6


/*
** (c)
*/

| ?- ke( [p->(q+r), p-> -s, s <-> q, -(p->r)] ).
   1     p -> q + r
   2     p -> - s
   3     s <-> q
   4     - (p -> r)
   5     alpha 4         p
   6     alpha 4         - r
   7     beta  2,5       - s
   8     beta  1,5       q + r
   9     beta  8,6       q
  10     eta   3,7       - q
         close 10,9

yes

| ?- ke( [p->(q+r), p-> -s, s <-> q, p->r] ).
   1     p -> q + r
   2     p -> - s
   3     s <-> q
   4     p -> r
   5     br-1  4              - p
   6          br-1  3              s
   7               eta   3,6       q
                   open [q, s, - p]
   8          br-2  3              - s
   9               eta   3,8       - q
                   open [- p, - q, - s]
  10     br-2  4              p
  11          beta  4,10      r
  12          beta  2,10      - s
  13          beta  1,10      q + r
  14          eta   3,12      - q
              open [p, r, - q, - s]

yes


Open branches, so keep saying yes
```

5. This question is about **KE**, Modal Logic and the Event Calculus. **KE**-trees (proofs) should be properly annotated to show the inference steps, i.e. showing the inference rule used, line number(s) of the premise(s), and the label for the world.

a) Using the Calculus **KE** for Modal Logic, prove that the following formula is valid in the logic **K**.

$$\Diamond(A \wedge B) \rightarrow (\Diamond A \wedge \Diamond B)$$

Using the Calculus **KE** for Modal Logic, prove that the following formula is *not* valid in the logic **K**.

$$(\Diamond A \wedge \Diamond B) \rightarrow \Diamond(A \wedge B)$$

Construct a Kripke Model as a counter-example and use it to explain the proof.

Explain whether or not the same results hold for logic **S5**.

[10]

b) Explain, with the use of an example, how knowledge is represented and reasoned with in:

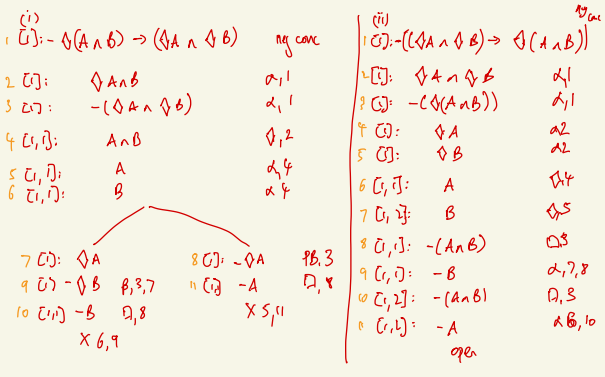
i) state-space formulation as a graph;

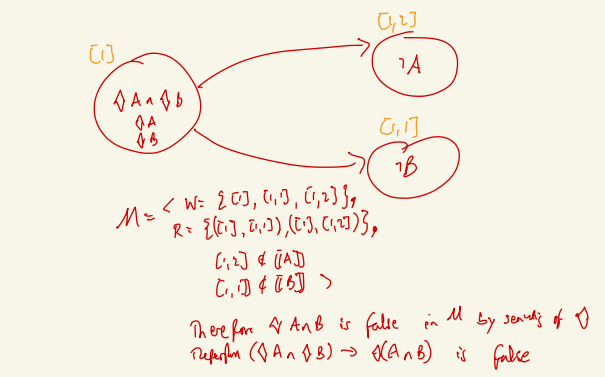
ii) a Kripke model; and

iii) axioms of the Event Calculus.

[10]

/* ** (a) */





Every theorem of K is included in S5, so yes.

Even if we made the counter-model reflective, symmetric and transitive by adding the appropriate links (extending the accessibility relation R), it would still be a counter-model for the formula, so yes.

/* ** (b) */

Take an example, eg farmer-wolf-goat-truss

Show how it is formulated in state space. Things of interest are modelled as elements of a tuple. Then we specify state change rules, and explicitly say what changes and what stays the same. The we can reason about sequences of actions, or plans.

Show how it is formulated as a Kripke model. Things of interest are modelled as propositions true or false in a possible. The state change rule is implicitly modelled by the accessibility relation. Then we reason about it with modal KE to prove properties of the system.

Show how it is formulated as axioms of the event calculus. Things of interest are modelled as fluent (ie propositions whose truth value changes over time). There are no explicit states, each fluent is a local state. State change rules only change fluent which are directly affected by the action, everything else stays the same. Can be used to do planning.

/*